

Linear Regression

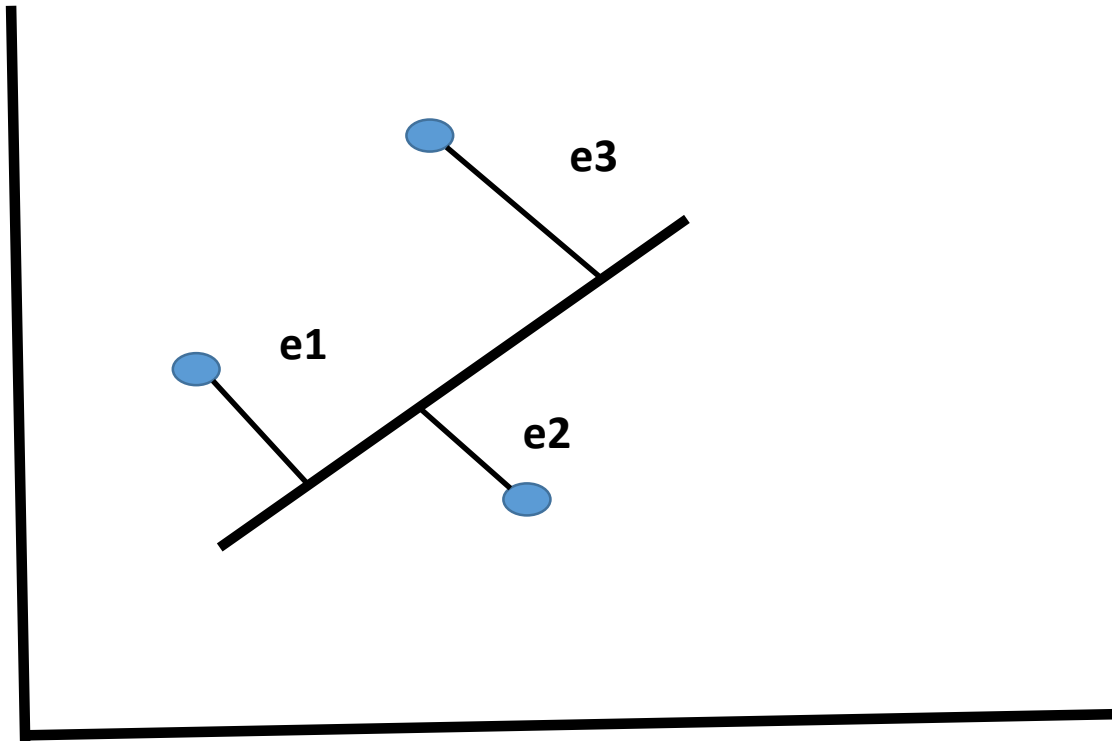
Linear Regression Example

- $y = a_0 + a_1x + \epsilon$
- a_0 intercept (Bias)
- a_1 Slope
- ϵ error in the prediction.

Assumption of Linear Regression

- 1) y is random & mutually independent
- 2) $\epsilon = |y_i - y_o|$
- y_i = Predicted output
- y_o = True output

Linear Regression Example



- $y_1 = (a_0 + a_1 x_1) + \epsilon_1$

- $y_2 = (a_0 + a_2 x_2) + \epsilon_2$

- .

- $y_n = (a_0 + a_n x_n) + \epsilon_n$

- $\epsilon_i = y_i - (a_0 + a_i x_i)$

- $\langle e_1, e_2, \dots, e_n \rangle$ are errors detailed

- $E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_i x_i))$

- As the minimisation of sum of absolute values of individual errors
- $E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_i x_i))$
- $E = \sum_{i=1}^n |e_i| = \sum_{i=1}^n |(y_i - (a_0 + a_i x_i))|$
- **Or sum of square**
- $E = \sum_{i=1}^n (e_i^2) = \sum_{i=1}^n ((y_i - (a_0 + a_i x_i))^2$
- So, Linear regression is modelled as a minimization function as follows
- $J(a, b) = \sum_{i=1}^n ((y_i - f(x_i))^2$
- $F(x_i) = (a_0 + a_i x_i)$
- Criteria function of parameter a_0 & a_i . These need to be minimized.
- Solution
- **Differentiate and zero**

- Differentiate and zero

- $a_1 = \frac{(\overline{xy}) - \bar{x} \cdot \bar{y}}{(\overline{x_i^2}) - (\bar{x})^2}$

- $a_0 = \bar{y} - a_1 \bar{x}$

Xi (Week)	Yi (Solution)	$\left((x_i)^2\right)$	$x_i \times y_i$
1	1.2	1	1.2
2	1.8	4	3.6
3	2.6	9	7.8
4	3.2	16	12.8
5	3.8	25	19
Sum=15 $\bar{x} = 15/5$ =3	Sum=12.6 $\bar{y} = 12.6/5$ =2.52	Sum=55 $\overline{(x_i)^2} = 55/5=11$	Sum= 44.4 $\bar{x} \bar{y} = 44.5/5$ 8.88

Xi (Week)	Yi (Solution)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

$$a_1 = \frac{(8.88 - 3 *2.52)}{11 -(3)^2} = \mathbf{0.66}$$

$$a_0 = 2.52 -0.66X3 = \mathbf{0.54}$$

$$\mathbf{Y= 0.54+0.66 \times x}$$

$$\mathbf{For\ x=7}$$

$$\mathbf{= 5.16}$$

Linear Regression in Matrix Form

$$\bullet \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix}$$

- $Y = Xa + e$,
- $X = n \times 2$
- $Y = n \times 1$
- $A = 2 \times 1$
- $e = n \times 1$

Xi (Week)	Yi (Prod Sales)
1	1
2	3
3	4
4	8

Xi (Week)	Yi (Prod Sales)
1	1
2	3
3	4
4	8

$$X^t = [1 \ 2 \ 3 \ 4], \quad Y^t = [1 \ 3 \ 4 \ 8]$$

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}$$

$$Y = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix}$$

$$\mathbf{a} = ((X^t X)^{-1} X^t) Y$$

After Calculating a,

a0 = Intercept = -1.5000

a1 = Slope = 2.2000

```
>> X'*X
```

```
ans =
```

```
4    10
```

```
10    30
```

```
>> inv(X'*X)
```

```
ans =
```

```
1.5000 -0.5000
```

```
-0.5000 0.2000
```

```
>> inv(X'*X)*X'
```

```
ans =
```

```
1.0000 0.5000 0 -  
0.5000
```

```
-0.3000 -0.1000 0.1000  
0.3000
```

```
>> (inv(X'*X)*X')*Y
```

```
ans =
```

```
-1.5000
```

```
2.2000
```

After calculating a,

a0 = Intercept = -1.5000

a1 = Slope = 2.2000

Y= -1.5 + 2.2X

Validation of Regression Model

xi	yi
I1	80
I2	90
I3	100
I4	110
I5	120

a) $MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$

• y_i = actual, $\hat{y}_i$ = Predicted

b) $MSE_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$

• Root Mean Square Error RMSE

c) $RMSE = \sqrt{MSE}$

d) Relative MSE

• $RelMSE = \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

0 – Model is perfect and its value between 0 to 1

1 not good

e) Coeff of CV

$$CV = \frac{RMSE}{\bar{y}}$$

Let I6 & I7 – Actual values are 80 & 75
Regression Predicts – 75 & 85

	yi	$\hat{y}_i$
I6	80	75
I7	75	85

$$\bar{y} = (80+90+100+110+120)/5 = 100$$

$$RelMSE = \frac{(80-75)^2 + (75-85)^2}{(80-100)^2 + (75-100)^2} = .1219$$

$$CV = .08$$

Validation of Regression Model

- a) $MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$
- y_i = actual, $\hat{y}_i$ = Predicted
- b) $MSE_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$
- Root Mean Square Error RMSE
- c) $RMSE = \sqrt{MSE}$
- d) Relative MSE
- $RelMSE = \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
- 0 – Model is perfect and its value between 0 to 1
- 1 not good
- e) Coeff of CV
- $$CV = \frac{RMSE}{\bar{y}}$$

xi	yi
I1	80
I2	90
I3	100
I4	110
I5	120

Let I6 & I7 – Actual values are 80 & 75
Regression Predicts – 75 & 85

	yi	$\hat{y}_i$
I6	80	75
I7	75	85

$$MAE = \frac{1}{2} * (|80-75| + |75-85|) = 15/2 = 7.5$$

$$MSE = 62.5$$

$$RMSE = \sqrt{62.5} = 7.91$$

Validation of Regression Model

Coeff of determination

$$r^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} = \sum (\hat{y}_i - \bar{y})^2$$

$$SS_{tot} = \sum (y_i - \hat{y})^2$$

$$r^2$$

1 to inf

Standard Error Estimation:

$$S_e = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n - 2}}$$

Multiple Linear Regression

- $Y = f(x_1, x_2)$

$$= a_0 + a_1 x_1 + a_2 x_2 + \dots + a_n x_n + \epsilon$$

Where $(x_1, \dots, x_n)$ are independent variables

Y is dependent

$(a_0, \dots, a_n)$ – Coef of regression coefficient

x1	x2	yi
1	4	1
2	5	6
3	8	8
4	2	12

X =

```

1  1  4
1  2  5
1  3  8
1  4  2

```

>> Y= [1 6 8 12]'

Y =

```

1
6
8
12

```

$$a = ((X^t X)^{-1} X^t) Y$$

>> a= (inv(X'*X)*X')*Y

a =

```

-1.6995
 3.4836
-0.0546

```

Bayesian Theorem

- Let the training dataset $\mathbf{D}$ consist of n points $\mathbf{x}_i$ in a d -dimensional space, and let y_i denote the class for each point,
- with $y_i \in \{c_1, c_2, \dots, c_k\}$
- The Bayes classifier directly uses the Bayes theorem to predict the class for a new test instance, $\mathbf{x}$.
- It estimates the posterior probability $P(c_i | \mathbf{x})$ for each class c_i , and chooses the class that has the largest probability.
- The predicted class for $\mathbf{x}$ is given as
- $\hat{y} = \mathop{\text{argmax}}_{c_i} \{ P(c_i | \mathbf{x}) \}$

Bayesian Theorem

- $\hat{y} = \mathop{\text{argmax}}_{c_i} \{ P(c_i | X) \}$
- The Bayes theorem allows us to invert the posterior probability in terms of the likelihood and prior probability, as follows:
- $$P(c_i | X) = \frac{P(X|c_i) \cdot P(c_i)}{P(X)}$$
- $P(X|c_i)$ is the *likelihood*, defined as probability of observing $\mathbf{x}$ assuming that the true class is c_i .
- $P(c_i)$ is the *prior probability* of class c_i , and $P(\mathbf{x})$ is the probability
- of observing $\mathbf{x}$ from any of the k classes, given as
- $$P(X) = \sum_{j=1}^k P(X|c_j) \cdot P(c_j)$$

Bayesian Theorem

$$\bullet \hat{y} = \mathop{arg}_{c_i}^{max} \{ P(c_i | X) \}$$

$$= \mathop{arg}_{c_i}^{max} \frac{P(X|c_i) \cdot P(c_i)}{P(X)}$$

$$= \mathop{arg}_{c_i}^{max} P(X|c_i) \cdot P(c_i)$$

- the predicted class essentially depends on the likelihood of that class
- taking its prior probability into account.
- For class c_i , the probability density at $\mathbf{x}$ is thus given as

$$f_i(\mathbf{x}) = f(\mathbf{x}|\mu_i, \Sigma_i) = \frac{1}{(\sqrt{2\pi})^d \sqrt{|\Sigma_i|}} \exp \left\{ -\frac{(\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)}{2} \right\}$$

BAYESCLASSIFIER ($\mathbf{D} = \{(\mathbf{x}_j, y_j)\}_{j=1}^n$):

```
1 for  $i = 1, \dots, k$  do  
2    $\mathbf{D}_i \leftarrow \{\mathbf{x}_j \mid y_j = c_i, j = 1, \dots, n\}$  // class-specific subsets  
3    $n_i \leftarrow |\mathbf{D}_i|$  // cardinality  
4    $\hat{P}(c_i) \leftarrow n_i/n$  // prior probability  
5    $\hat{\boldsymbol{\mu}}_i \leftarrow \frac{1}{n_i} \sum_{\mathbf{x}_j \in \mathbf{D}_i} \mathbf{x}_j$  // mean  
6    $\mathbf{Z}_i \leftarrow \mathbf{D}_i - \mathbf{1}_{n_i} \hat{\boldsymbol{\mu}}_i^T$  // centered data  
7    $\hat{\boldsymbol{\Sigma}}_i \leftarrow \frac{1}{n_i} \mathbf{Z}_i^T \mathbf{Z}_i$  // covariance matrix  
8 return  $\hat{P}(c_i), \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i$  for all  $i = 1, \dots, k$ 
```

TESTING ($\mathbf{x}$ and $\hat{P}(c_i), \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i$, for all $i \in [1, k]$):

```
9  $\hat{y} \leftarrow \arg \max_{c_i} \{ f(\mathbf{x} | \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i) \cdot P(c_i) \}$   
10 return  $\hat{y}$ 
```

Example

- Consider the 2-dimensional Iris data, with attributes sepal length and sepal width. Class c_1 , which corresponds to iris-setosa has $n_1 = 50$ points, whereas the other class c_2 has $n_2 = 100$ points.

The prior probabilities for the two classes are

$$\hat{P}(c_1) = \frac{n_1}{n} = \frac{50}{150} = 0.33$$

$$\hat{P}(c_2) = \frac{n_2}{n} = \frac{100}{150} = 0.67$$

$$\hat{\mu}_1 = \begin{pmatrix} 5.01 \\ 3.42 \end{pmatrix}$$

$$\hat{\mu}_2 = \begin{pmatrix} 6.26 \\ 2.87 \end{pmatrix}$$

$$\hat{P}(c_1) = \frac{n_1}{n} = \frac{50}{150} = 0.33$$

$$\hat{P}(c_2) = \frac{n_2}{n} = \frac{100}{150} = 0.67$$

$$\hat{\mu}_1 = \begin{pmatrix} 5.01 \\ 3.42 \end{pmatrix}$$

$$\hat{\mu}_2 = \begin{pmatrix} 6.26 \\ 2.87 \end{pmatrix}$$

$$\hat{\Sigma}_1 = \begin{pmatrix} 0.122 & 0.098 \\ 0.098 & 0.142 \end{pmatrix}$$

$$\hat{\Sigma}_2 = \begin{pmatrix} 0.435 & 0.121 \\ 0.121 & 0.110 \end{pmatrix}$$

Let $\mathbf{x} = (6.75, 4.25)^T$ be a test point (shown as white square).

The posterior probabilities for c_1 and c_2 can be computed using

$$\hat{P}(c_1|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_1, \hat{\Sigma}_1) \hat{P}(c_1) = (4.914 \times 10^{-7}) \times 0.33 = 1.622 \times 10^{-7}$$

$$\hat{P}(c_2|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_2, \hat{\Sigma}_2) \hat{P}(c_2) = (2.589 \times 10^{-5}) \times 0.67 = 1.735 \times 10^{-5}$$

Because $\hat{P}(c_2|\mathbf{x}) > \hat{P}(c_1|\mathbf{x})$ the class for $\mathbf{x}$ is predicted as $\hat{y} = c_2$.

Challenges

- for numeric attributes we have to estimate $O(d^2)$ covariances, and as the dimensionality increases, this requires us to estimate too many parameters.

NAIVE BAYES CLASSIFIER

- The naive Bayes approach makes the simple assumption that all the attributes are independent.
- Training the naive Bayes classifier is very fast, with $O(nd)$ computational complexity.

NAIVE BAYES CLASSIFIER

$$\hat{\Sigma}_1 = \begin{pmatrix} 0.122 & 0 \\ 0 & 0.142 \end{pmatrix}$$

$$\hat{\Sigma}_2 = \begin{pmatrix} 0.435 & 0 \\ 0 & 0.110 \end{pmatrix}$$

For the test point $\mathbf{x} = (6.75, 4.25)^T$, the posterior probabilities for c_1 and c_2 are as follows:

$$\hat{P}(c_1|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_1, \hat{\Sigma}_1) \hat{P}(c_1) = (3.99 \times 10^{-7}) \times 0.33 = 1.32 \times 10^{-7}$$

$$\hat{P}(c_2|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_2, \hat{\Sigma}_2) \hat{P}(c_2) = (9.597 \times 10^{-5}) \times 0.67 = 6.43 \times 10^{-5}$$

Because $\hat{P}(c_2|\mathbf{x}) > \hat{P}(c_1|\mathbf{x})$ the class for $\mathbf{x}$ is predicted as $\hat{y} = c_2$.

NAIVE BAYES CLASSIFIER

$$\mathbf{\Sigma}_i = \begin{pmatrix} \sigma_{i1}^2 & 0 & \dots & 0 \\ 0 & \sigma_{i2}^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \dots & \sigma_{id}^2 \end{pmatrix}$$

This yields

$$|\mathbf{\Sigma}_i| = \det(\mathbf{\Sigma}_i) = \sigma_{i1}^2 \sigma_{i2}^2 \cdots \sigma_{id}^2 = \prod_{j=1}^d \sigma_{ij}^2$$

Also, we have

$$\mathbf{\Sigma}_i^{-1} = \begin{pmatrix} \frac{1}{\sigma_{i1}^2} & 0 & \dots & 0 \\ 0 & \frac{1}{\sigma_{i2}^2} & \dots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \dots & \frac{1}{\sigma_{id}^2} \end{pmatrix}$$

Training dataset

Class:

C1:buys_computer='yes'

C2:buys_computer='no'

Data sample

X =(age<=30,

Income=medium,

Student=yes

Credit_rating=Fair)

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
30...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

- Let C_1 correspond to the class buys computer = yes and C_2 correspond to buys computer = no. The unknown sample we wish to classify is
- $X = (\text{age} = "<30", \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit rating} = \text{fair})$
- We need to maximize $P(X|C_i)P(C_i)$, for $i = 1, 2$. $P(C_i)$, the prior probability of each class, can be computed based on the training samples:
- $P(\text{buys computer} = \text{yes}) = 9/14 = 0.643$
- $P(\text{buys computer} = \text{NO}) = 5/14 = 0.357$

Naïve Bayesian Classifier: Example

- Compute $P(X/C_i)$ for each class

$$P(\text{age} = "<30" \mid \text{buys_computer} = \text{"yes"}) = 2/9 = 0.222$$

$$P(\text{age} = "<30" \mid \text{buys_computer} = \text{"no"}) = 3/5 = 0.6$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"yes"}) = 4/9 = 0.444$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"no"}) = 1/5 = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

K NEAREST NEIGHBORS CLASSIFIER

- **non-parametric approach,**
- which does not make any assumptions about the underlying joint probability density function.
- it directly uses the data sample to estimate the density,
- Let **$\mathbf{D}$** be a training dataset comprising n points $\mathbf{x}_i \in \mathbb{R}^d$,
- and let **$\mathbf{D}_i$** denote the subset of points in **$\mathbf{D}$** that are labeled with class c_i , with $n_i = |\mathbf{D}_i|$.
- Given a test point $\mathbf{x} \in \mathbb{R}^d$, and K , the number of neighbors to consider,
- let r denote the distance from $\mathbf{x}$ to its K th nearest neighbor in **$\mathbf{D}$** .

***K* NEAREST NEIGHBORS CLASSIFIER**

- Consider the d -dimensional hyperball of radius r around the test point $\mathbf{x}$, defined as
- $\mathbf{B}_d(\mathbf{x}, r) = \{\mathbf{x}_i \in \mathbf{D} \mid \delta(\mathbf{x}, \mathbf{x}_i) \leq r\}$
- $\delta(\mathbf{x}, \mathbf{x}_i)$ is the distance between $\mathbf{x}$ and $\mathbf{x}_i$, which is usually assumed to be the Euclidean distance,
- $\delta(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2$
- Let K_i denote the number of points among the K nearest neighbors of $\mathbf{x}$ that are labeled with class c_i ,

$$K_i = \#\{ \mathbf{x}_j \in \mathbf{B}_d(\mathbf{x}, r) \mid \mathbf{y}_j = c_i \}$$

K NEAREST NEIGHBORS CLASSIFIER

- The class conditional probability density at $\mathbf{x}$ can be estimated as the fraction of points from class c_i that lie within the hyperball divided by its volume.

$$\hat{f}(\mathbf{x}|c_i) = \frac{K_i/n_i}{V} = \frac{K_i}{n_i V}$$

- where $V = \text{vol}(B_d(\mathbf{x}, r))$ is the volume of the d -dimensional hyperball.
- posterior probability $P(c_i | \mathbf{x})$ can be estimated as

$$P(c_i|\mathbf{x}) = \frac{\hat{f}(\mathbf{x}|c_i) \hat{P}(c_i)}{\sum_{j=1}^k \hat{f}(\mathbf{x}|c_j) \hat{P}(c_j)}$$

K NEAREST NEIGHBORS CLASSIFIER

- However, because $\hat{P}(c_i) = n_i / n$, we have

$$\hat{f}(\mathbf{x}|c_i)\hat{P}(c_i) = \frac{K_i}{n_i V} \cdot \frac{n_i}{n} = \frac{K_i}{nV}$$

Thus the posterior probability is given as

$$P(c_i|\mathbf{x}) = \frac{\frac{K_i}{nV}}{\sum_{j=1}^K \frac{K_j}{nV}} = \frac{K_i}{K}$$

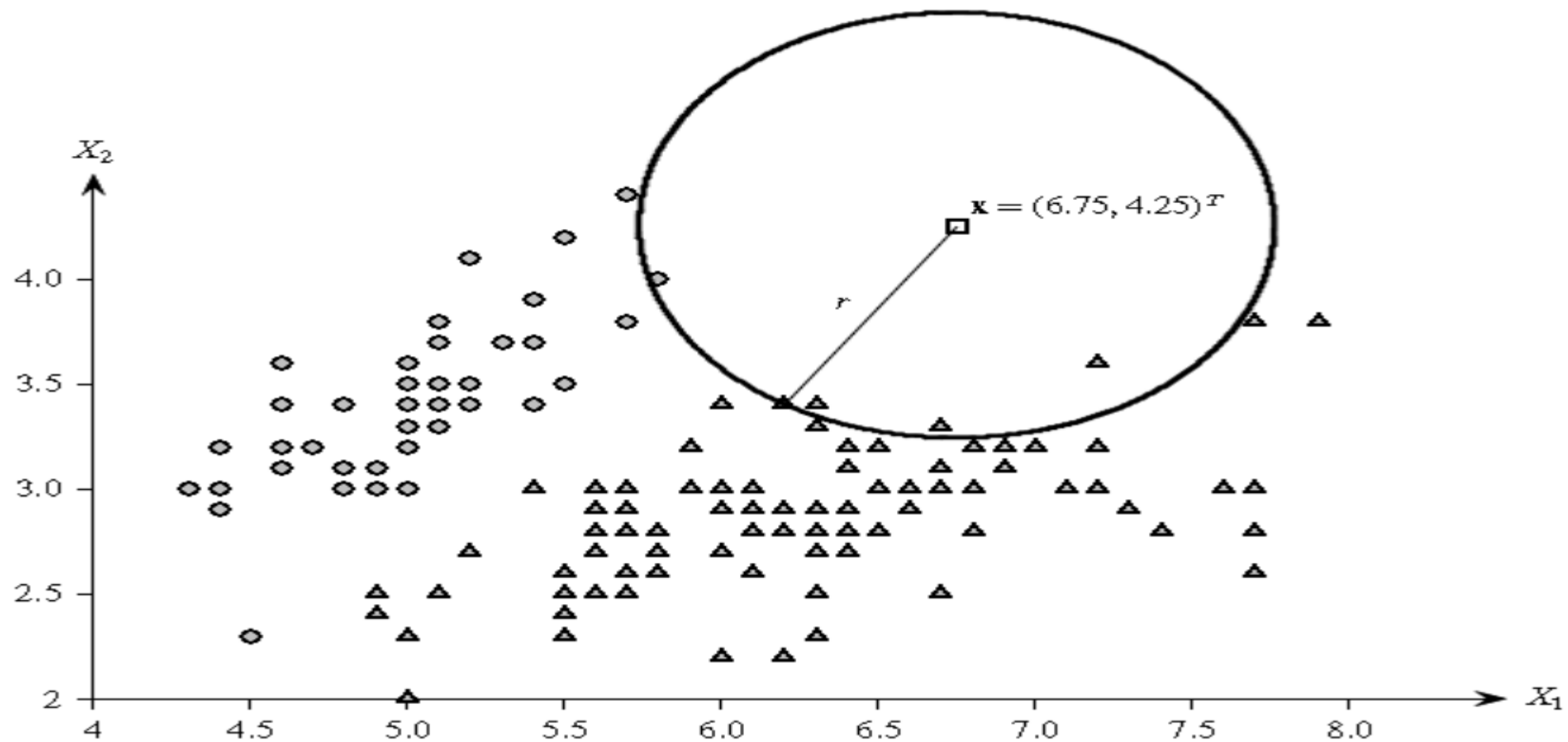
Finally, the predicted class for $\mathbf{x}$ is

$$\hat{y} = \arg\max_{c_i} \{P(c_i|\mathbf{x})\} = \arg\max_{c_i} \left\{ \frac{K_i}{K} \right\} = \arg\max_{c_i} \{K_i\}$$

Example

- Consider the 2D Iris dataset . The two classes are: c_1 (circles) with $n_1 = 50$ points and c_2 (triangles) with $n_2 = 100$ points.
- Let us classify the test point $\mathbf{x} = (6.75, 4.25)^T$ using its $K = 5$ nearest neighbors, is given as $r = \sqrt{1.025} = 1.012$
- It encompasses $K_1 = 1$ point from class c_1
- and $K_2 = 4$ points from class c_2 .
- **Therefore, the predicted class for $\mathbf{x}$ is $\hat{y} = c_2$.**

Example



Capacity, Overfitting and Underfitting

- The ability to perform well on previously unobserved inputs is called **generalization**.
- when training a machine learning model, we have access to a **training set**, we can compute some error measure on the training set called the **training error**, and we **reduce this training error**.
- This is an **optimization problem**.
- What separates machine learning from optimization is that we want the **generalization error, also called the test error, to be low** as well.
- **generalization error** is the expected value of error on a **new input**

Capacity, Overfitting and Underfitting

- we trained the model by minimizing the training error,
- $\frac{1}{m^{(train)}} \|X^{(train)}\mathbf{w} - \mathbf{y}^{(train)}\|_2^2$

but we actually care about the test error,

- $\frac{1}{m^{(test)}} \|X^{(test)}\mathbf{w} - \mathbf{y}^{(test)}\|_2^2$

How can we affect performance on the test set when we get to observe only the training set?

Capacity, Overfitting and Underfitting

- The train and test data are generated by a probability distribution over datasets called the data generating process.
- We typically make a set of assumptions known collectively as the **i.i.d. assumptions**.
- These assumptions are that the examples in each dataset are independent from each other,
- and that the **train set and test set** are **identically distributed**, drawn from the same probability distribution as each other.

Capacity, Overfitting and Underfitting

- The factors determining how well a machine learning algorithm will perform are its ability to:
 1. **Make the training error small.**
 2. **Make the gap between training and test error small.**
- These two factors correspond to the two central challenges in machine learning:
- **underfitting and overfitting .**
- **Underfitting occurs** when the model is not able to obtain a sufficiently **low error value on the training set.**
- **Overfitting occurs** when the gap between the **training error and test error** is too large.

Capacity, Overfitting and Underfitting

- A model is said to be a good machine learning to make predictions about future data.
- overfitting and underfitting, which are majorly responsible for the **poor performances**.

underfitting :

- a model is too simple to capture data complexities.
- inability of the model to learn the training data effectively result in poor performance **both on the training and testing data**.
- Models are inaccurate
- size of the training dataset used is not enough.

Overfitting

- model does not make accurate predictions on testing data.
- When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set.

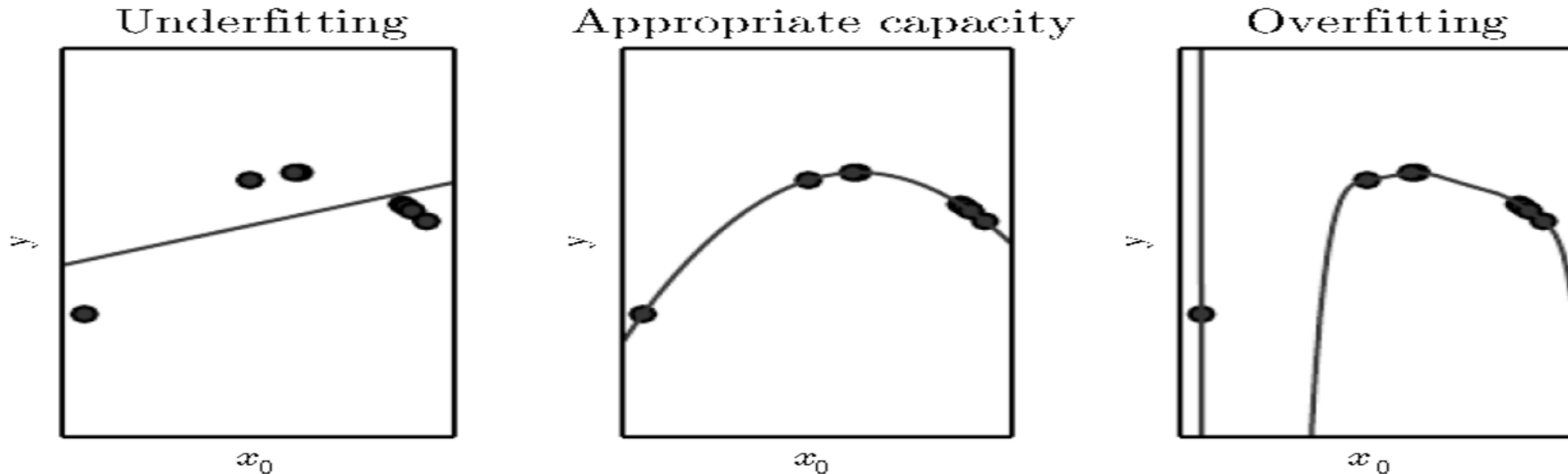
Capacity, Overfitting and Underfitting

- We can control whether a model is more likely to overfit or underfit by **altering its capacity**.
- Models with **low capacity** may struggle to fit the training set.
- Models with **high capacity** can overfit by memorizing properties of the training set that do not serve them well on the test set.
- We can generalize linear regression to include polynomials, rather than just linear functions, in its hypothesis space. Doing so increases the model's capacity.
- $\hat{y} = b + wx$
- $\hat{y} = b + w_1x + w_2x^2$
- obtain a polynomial of degree 9:
- $\hat{y} = b + \sum_{i=1}^9 w_i x^i$

Capacity, Overfitting and Underfitting

- Models with insufficient capacity are unable to solve complex tasks.
- Models with high capacity can solve complex tasks,
 - but when their capacity is higher than they may overfit.
- We compare a linear, quadratic and degree-9 predictor attempting to fit a problem where the true underlying function is quadratic.
- The linear function is **unable to capture the curvature** in the true underlying problem, so it underfits.
- The **degree-9 predictor** is capable of representing the correct function, but it is also capable of representing infinitely many other functions that pass exactly through the training points,

Capacity, Overfitting and Underfitting



(Left) A linear function fit to the data suffers from underfitting—it cannot capture the curvature that is present in the data.

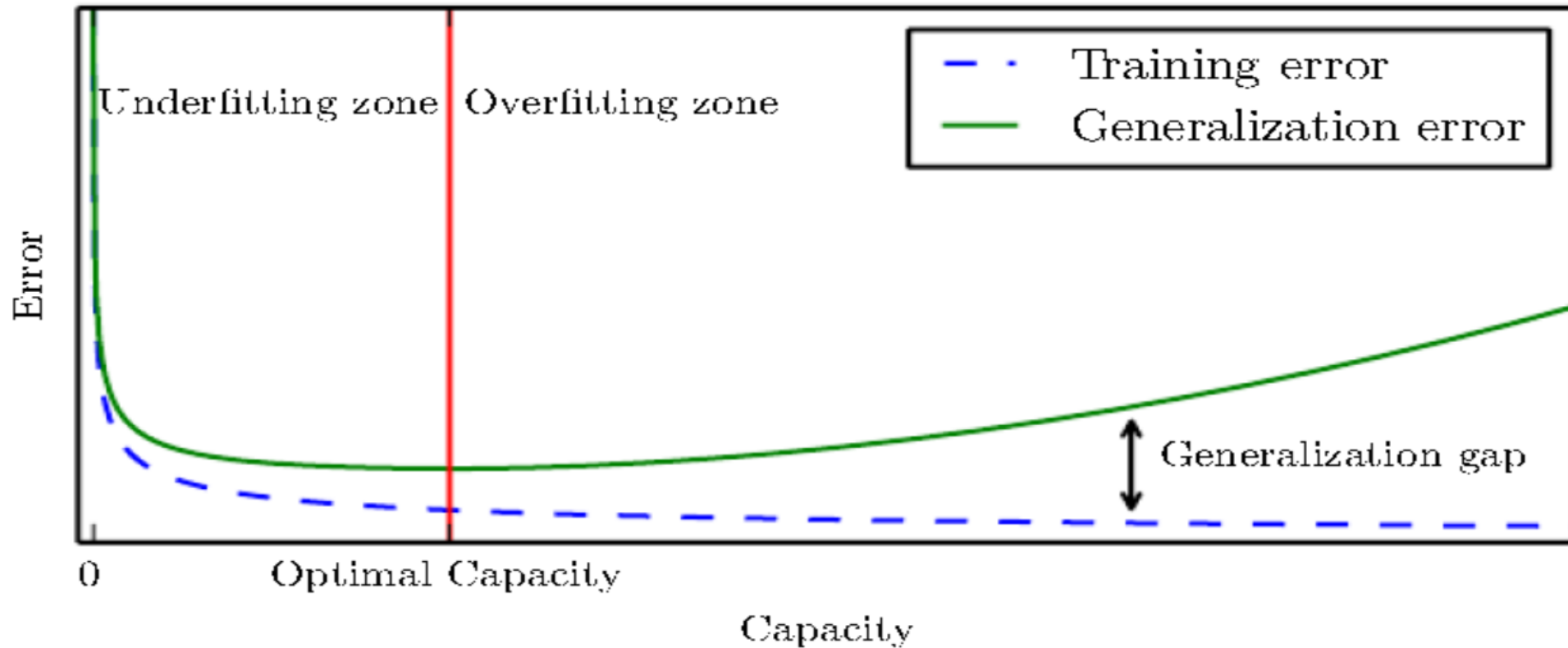
(Center) A quadratic function fit to the data generalizes well to unseen points. It does not suffer from a significant amount of overfitting or underfitting.

(Right) A polynomial of degree 9 fit to the data suffers from overfitting

Capacity, Overfitting and Underfitting

- one way of changing a model's capacity:
- by changing the number of input features it has,
- and simultaneously adding new parameters associated with those features.
- In many cases, finding the best function within this family is a very difficult optimization problem.
- In practice, the learning algorithm does not actually find the best function, but merely one that significantly reduces the training error.

Capacity, Overfitting and Underfitting



Capacity, Overfitting and Underfitting

- Training and test error behave differently.
- At the left end of the graph, **training error and generalization error are both high. This is the underfitting regime.**
- **As we increase capacity, training error decreases, but the gap between training and generalization error increases.**

Capacity, Overfitting and Underfitting

- **Non-parametric models have no such limitation.**
- One example of such an algorithm is **nearest neighbor regression**.
- Unlike linear regression, which has a fixed-length vector of weights,
- the nearest neighbor regression model simply stores the **X and y from the training set**.
- When asked to classify a test point x , the model looks up the nearest entry in the training set and returns the associated regression target.
- $\hat{y} = y_i$ **where $i = \arg \min \|X_i - x\|_2^2$**

Capacity, Overfitting and Underfitting

- The ideal model is an **oracle** that simply knows the true **probability distribution that generates the data**.
- Even such a model will still incur some error on many problems, because there may still be **some noise in the distribution**.

In the case of supervised learning, the mapping from x to y may be inherently stochastic, or y may be a deterministic function that involves other variables besides those included in x .

The error incurred by an oracle making predictions from the true distribution $p(x, y)$ is called the **Bayes error**.

Capacity, Overfitting and Underfitting

- Training and generalization error vary as the size of the training set varies.
- Expected generalization error can never increase as the number of training examples increases.
- **For non-parametric models, more data yields better generalization until the best possible error is achieved.**

Any fixed parametric model with less than optimal capacity will asymptote to an **error value** that exceeds the Bayes error.

- it is possible for the model to have optimal capacity and yet still have a large gap between training and generalization error.
- we may be able to reduce this gap by gathering more training examples.

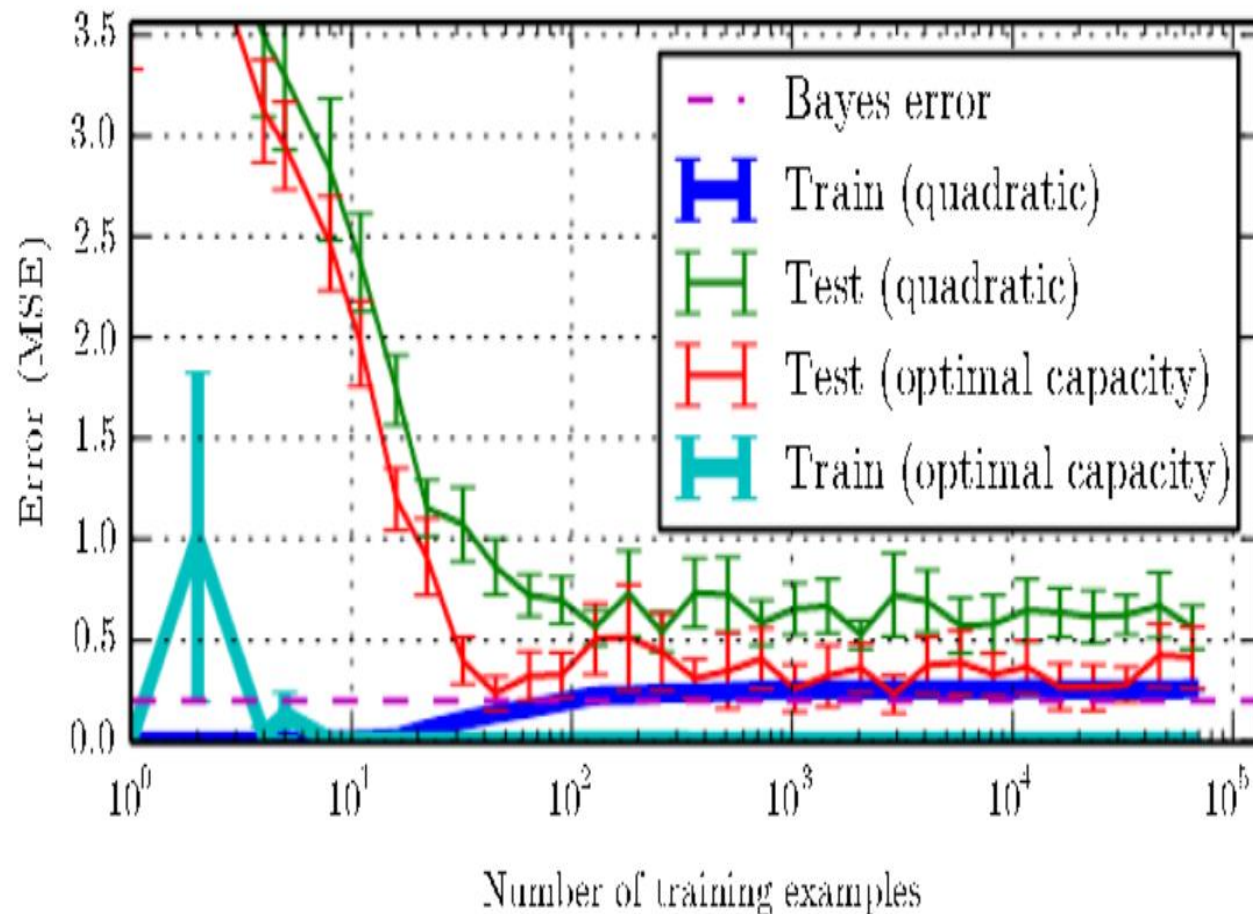
Capacity, Overfitting and Underfitting

- nearest neighbor regression model simply stores the X and y from the training set.
- When asked to classify a test point x , the model looks up the nearest entry in the training set and returns the associated regression target.
- In other words, $\hat{y} = y_i$ where $i = \arg \min ||X_i - x||_2^2$
- The error incurred by an oracle making predictions from the true distribution $p(x, y)$ is called the **Bayes error**.
- Expected generalization error can never increase as the number of training examples increases.
- For non-parametric models, more data yields better generalization until the best possible error is achieved.

Capacity, Overfitting and Underfitting

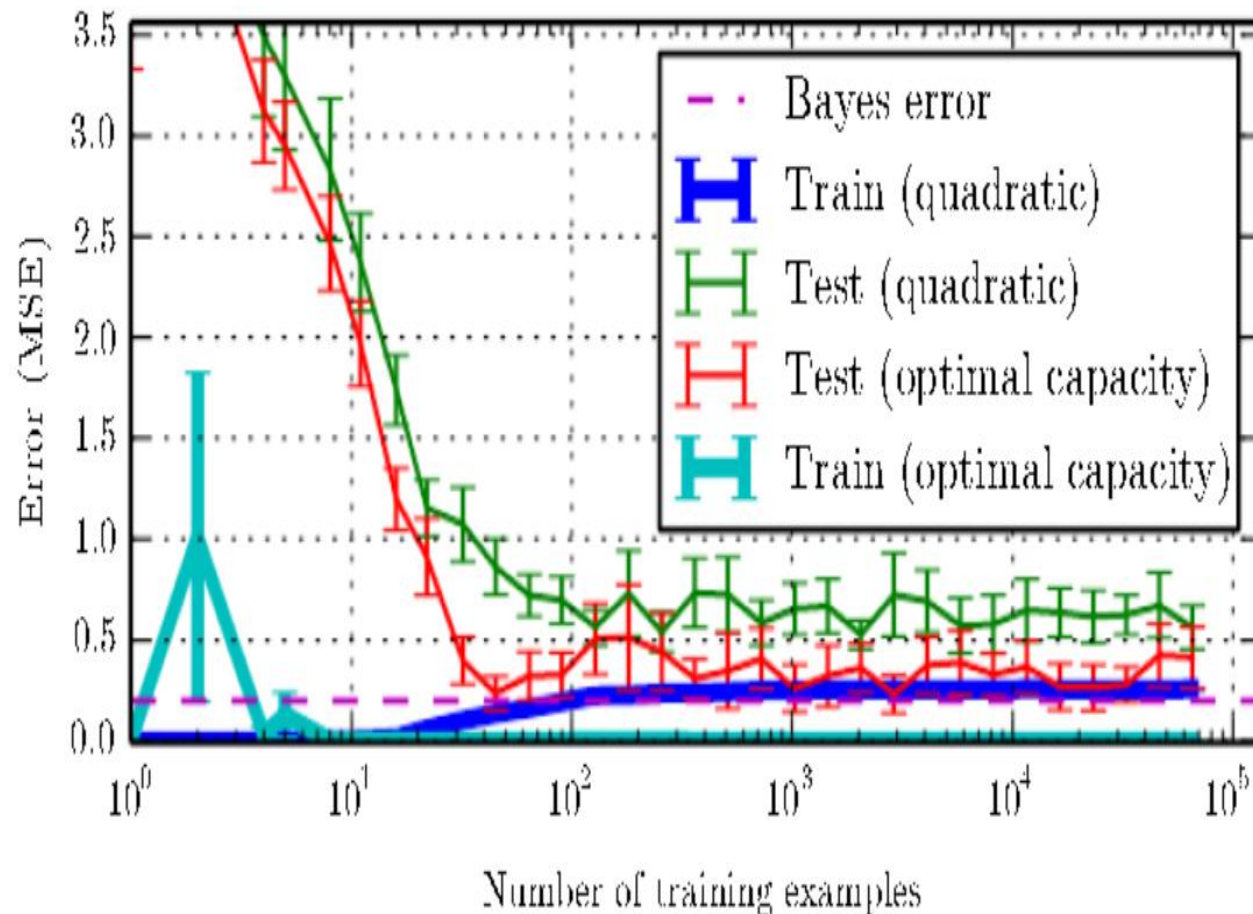
- it is possible for the model to have **optimal capacity** and yet still have a large gap between training and generalization error.
- we may be able to reduce this gap by gathering more training examples.
- **The No Free Lunch Theorem**
- every classification algorithm has the same error rate when classifying previously unobserved points.
- **no machine learning algorithm is universally any better than any other.**

Capacity, Overfitting and Underfitting



For the quadratic model,
training error increases as size of the training set increases.
test error decreases, because fewer incorrect hypotheses are consistent with the training data.
The quadratic model **does not have enough capacity** to solve the task, so its test error asymptotes to a **high value**.

Capacity, Overfitting and Underfitting



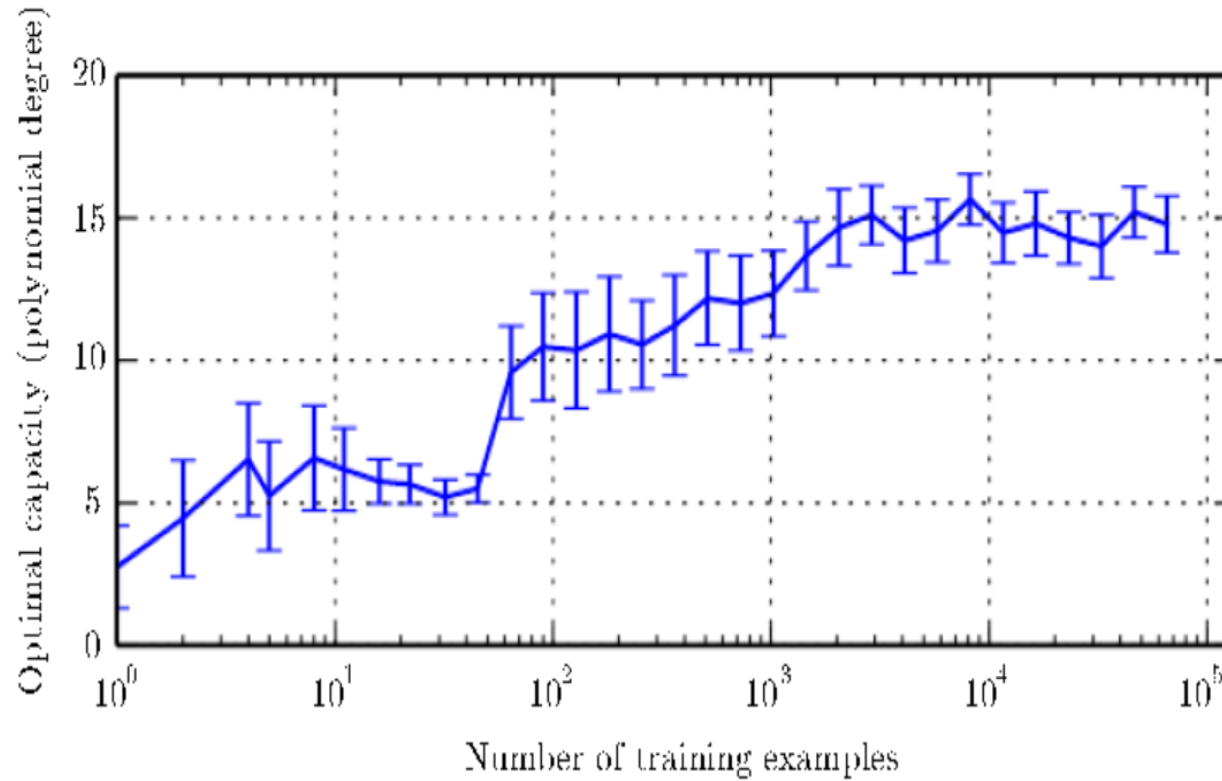
test error at optimal capacity

asymptotes to Bayes error.

training error can fall below **Bayes error**,

As **training size** increases to **infinity**, **training error** of any fixed-capacity model (**quadratic model**) must rise to at least Bayes error

Capacity, Overfitting and Underfitting



As the training set size increases, the optimal capacity (degree of the optimal polynomial regressor) increases.

goal of machine learning research is not to seek a universal learning algo or absolute best learning algo. Instead, our goal is to understand what kinds of distributions are relevant to the “real world” .